Avance preliminar proyecto física computacional

Antonia Morales

[1] *Departamento de Física, Universidad Técnica Federico Santa María, San Joaquín, Chile*

Introducción

Este proyecto busca simular computacionalmente la producción de muones atmosféricos y su posterior detección y análisis, cabe destacar que el proceso de generación de muones fue simplificado únicamente a la generacion de una energía inicial, y un vector de posición inicial que sean consistentes con la física del proceso, sin embargo se omiten otras variables que determinan algunas de las cantidades físicas de interés como por ejemplo la producción en sí de los muones a partir de un pion o un kaon.

1. Funciones implementadas

1.1. Funciones generadoras de ángulos

```
def theta():
    u = np.random.rand() # genero n randoms entre 0 y 1
    return np.arccos(u**(1/3))

def phi():
    return np.random.rand()* 2*np.pi
```

La función **theta** genera un ángulo cenital aleatorio que sigue una distribución como $\cos^2\theta$. Para conseguir la distribución deseada se utiliza la función de probabilidad acumulada y se introduce la variable auxiliar u que será igual a esta función, luego se despeja θ en función de u.

La función **phi** genera un ángulo aleatorio entre 0 y 2π el cual será usado como ángulo azimutal.

1.2. función de vector

```
def vector(theta,phi): # los angulos creados anteriormente

    vx = np.sin(theta) * np.cos(phi)
    vy = np.sin(theta) * np.sin(phi)
    vz = np.cos(theta)

    return np.array([vx, vy, vz])
```

Esta función genera un vector en coordenadas esféricas con los ángulos. Luego será utilizada para medir distancias y tiempos de vuelo.

1.3. Energía inicial

```
def energia_inicial(Emin=100, Emax=100000.0, gamma=2.7): # en Mev
    u = np.random.rand()
    a = 1 - gamma
    E0 = ((Emax**a - Emin**a)*u + Emin**a)**(1/a)
    return E0
```

Se incia una energía que sigue una distrubución tal que $\frac{dN}{dE} = E^{-2,7}$ [**1**], esto siendo una aproximación del espectro de energía real el cual depende del espectro de los piones que los producen, esta funcion retorna el valor de la energia que tendrá un muon que llegue al detector

1.4. Perdida de energía en el detector

```
def bethe_bloch(E_MeV):

    if E_MeV < 0.1: return 0.0
    m_mu = 105.658 ; me=0.511; K=0.307075
    Z_A=0.5551; rho=1.03; I=75e-6
    gamma = 1 + E_MeV/m_mu
    beta2 = 1 - 1/gamma**2
    Tmax  = (2*me*beta2*gamma**2)/(1+2*gamma*me/m_mu+(me/m_mu)**2)
    arg   = 2*me*beta2*gamma**2*Tmax/I**2
    if arg<=1 or beta2<=0: return 0.0
    return max(K*rho*Z_A/beta2*(0.5*np.log(arg)-beta2), 0.0)


def perdida(E0, theta, X0, dx=0.5):
    X = X0 / np.cos(theta) # distancia recorrida
    E = E0
    x=0

    while x<X and E>0:
        E -= bethe_bloch(E)*dx
        x += dx
    return E
```

La primera función retorna dE/dx o la perdida de energía por unidad de longitud según el modelo de Bethe-Bloch y la segunda calcula dE/dx según la energía local y resta la pérdida.

1.5. Generador de muones

```
def generar_muon (): # energia en MeV
    E0 = energia_inicial()
    th= theta()
    ph = phi()
    v = vector(th, ph)
    X0 = 10 #cm espesor del detector ???

    # llegada al detector
    L =X0/np.cos(th)
    E =  perdida(E0, th, X0)
    # energia con la que el muon llega al detector
    E_dep =  E0-E

    return {
        "E0": E0,
        "theta": th,
        "phi": ph,
        "E_dep": E_dep
    }
```

Esta función genera muon con una energía inicial al llegar al detector, y sus ángulos de posicion, asi como su vector de posición, tambien se define el espesor del detector y se calcula la energía perdida por bethe bloch dentro del detector, posteriormente se calcula la energía depositada en el centellador para usarla para calcular la señal del fotomultiplicador.

1.6. fotomultiplicador

```
def fotomultiplicador(E_dep, g= 10e6, s = 0.03):
   mu_pe =  E_dep * 10000*0.25 #valor esperado de fotoelectrones
   N_pe = np.random.poisson(mu_pe) #numero real de fotoelectrones
   Q = N_pe * g  # carga acumulada en ánodo luego de multiplicación
   sigma = np.sqrt(max(N_pe,1)) * g * s * 1.602e-19
   Q = np.random.normal(Q, sigma)

   return Q
```

La función fotomultiplicador recibe la energía depositada en el centellador. Se asume que por cada MeV de energía depositada, se producen 10000 fotones y además se asume una eficiencia cuántica de 25 % es decir, la probabilidad de que un fotón incidente sobre el fotocátodo emita un fotoelectrón.
Por otra parte se asume que el pmt tiene una ganancia de 10^6, en otras palabras por cada fotoelectron emitido en el fotocátodo, se recibirán 10^6 electrones en el ánodo, finalmente se retorna la carga recibida en el ánodo la cual nos será de utilidad para posteriormente convertir la información a una señal eléctrica

2. Avances futuros

- Conversión de carga acumulada en el ánodo del pmt a señal eléctrica.
- detección de umbral
- lógica de trigger por coincidencias entre capas

- reconstrucción del ángulo de incidencia por ToF
- distribución angular reconstruida vs. entrada $\cos^2\theta$
- flujo de muones vs. altitud.

3. Referencias

1 Geisser. Cosmic rays and particle physics.

2 P.A. Zyla et al. (Particle Data Group), *Cosmic Rays*, 2020. Disponible en: https://pdg.web.cern.ch/pdg/2020/reviews/rpp2020-rev-cosmic-rays.pdf

3 S. Almehed et al., *ATLAS TileCal: The Tile Calorimeter Technical Design Report*, CERN-LHCC-96-042, 1999. Disponible en: https://cds.cern.ch/record/683884/files/tilecal-99-012.pdf